

Bayesian workflow

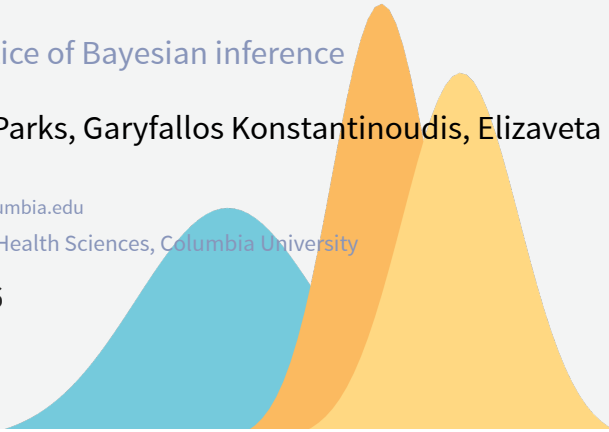
Good practice of Bayesian inference

Robbie M. Parks, Garyfallos Konstantinoudis, Elizaveta Semenova

robbie.parks@columbia.edu

Environmental Health Sciences, Columbia University

Aug 5, 2026



Bayesian inference

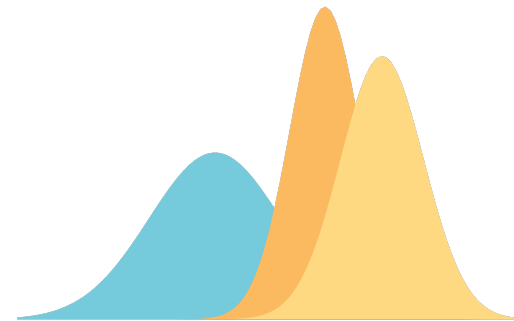
Outline

- Bayesian inference recap: What? Why? How?
- What is a Bayesian workflow and why do we need it?
- Principles of Bayesian workflow.
- Modern Bayesian workflow.
- Getting ready for the lab.

Bayes formula:

$$p(\theta|y) = \frac{p(y|\theta)p(\theta)}{p(y)}$$

Can you recall what the components of the Bayes formula are?



Bayes formula:

$$p(\theta|y) = \frac{p(y|\theta)p(\theta)}{p(y)}$$

Can you recall what the components of the Bayes formula are?



- $p(\theta)$ is the *prior* distribution, i.e. what is known *a priori*.
- $p(y|\theta)$ is the *likelihood*, i.e. probability of observing the data given parameters θ .
- $p(\theta|y)$ is the *posterior* distribution, i.e. the distribution of parameters of interest after data were observed.

General principle of Bayesian inference:

- Specify a complete Bayesian model:
 - Specify likelihood.
 - Specify priors for all parameters (saying 'I don't know' is also a prior).

Bayes rule:

$$\underbrace{p(\theta|y)}_{\text{posterior}} \propto \underbrace{p(y|\theta)}_{\text{likelihood}} \underbrace{p(\theta)}_{\text{prior}}$$

What can possibly go wrong?

A lot can go wrong!

General principle of Bayesian inference:

- Example:
 - Consider **data** $y = \{y_1, \dots, y_n\}$
 - Specify an **observation model**, e.g.

$$p(y|\theta) = \prod_i N(y_i|\theta, \sigma^2)$$

- complete the model with a prior distribution, e.g.

$$p(\theta) = N(0, 1)$$

General principle of Bayesian inference:

- Specify a complete Bayesian model.
- Sample the posterior distribution of the parameter θ .

Sometimes posterior is available in a closed form.

- Examples of *conjugate* models include Normal-Normal, Poisson-Gamma, and Binomial-Beta, as we saw this morning.

Remark on inference methods

- Alongside sampling methods such as MCMC, there are also approximate methods, such as Variational Inference (VI) and [INLA](#).
- MCMC is the more general of the two: it applies to almost any model, and given enough time the chain will *converge*.
- Approximate methods trade that generality for speed. [INLA](#) applies only to latent Gaussian models, but within that class it is deterministic and often much faster.
- For today we use MCMC, which is why the workflow below is built around chain diagnostics.

Probabilistic programming languages

Probabilistic programming languages (PPLs) from a user's perspective:

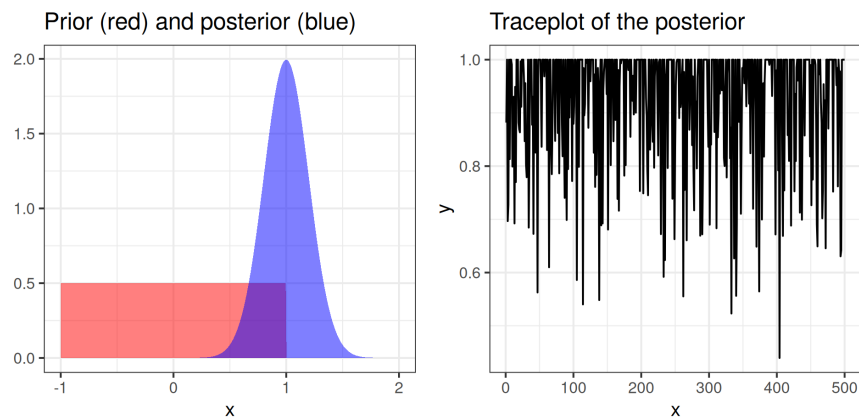
- PPLs are designed to let the user **focus on modelling** while inference happens automatically.
- Users need to specify:
 - Prior.
 - Likelihood.
- Inference is performed via powerful algorithms such as, Markov Chain Monte Carlo (MCMC).
- Availability of **diagnostic tools**.

Principles of Bayesian workflow

Some generic checks

- Priors:
 - How to select a prior? (partly philosophical)
 - Is our prior compatible with the data? (prior predictive checks)
- Posterior (or MCMC output):
 - Convergence diagnostics (traceplots, autocorrelation plots, etc.)
 - Model fit (posterior predictive checks)
 - Others: Model predictive ability (cross-validation), model selection (information criteria)

Example 1: The prior is too strict

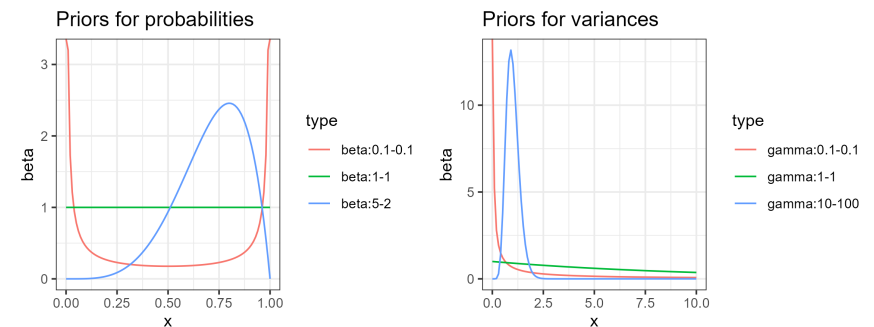


How to select a prior?

- The prior should allow adequate exploration of the parameter space of the random variable.
- The prior should be compatible with the nature of the random variable.
- The priors should reflect our prior knowledge.

Example 2: Respect the nature of the random variable

How should we model probabilities and variances?



Priors should reflect our prior knowledge

I know that I know nothing. Socrates

- If you do know, then priors should reflect your knowledge.
 - Treatment effects for numerous previous studies or meta-analysis.
 - How smooth an effect can be. (we will discuss later)
- If you do not know, then the priors should reflect your lack of knowledge.
 - Vague priors, non-informative priors.

Examples

Prior predictive checks

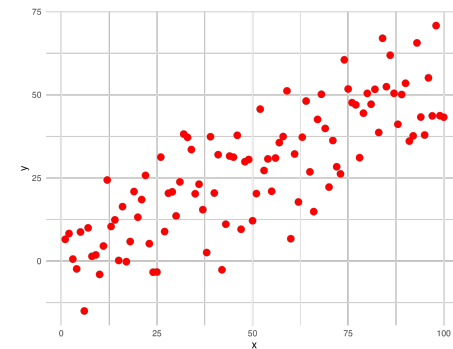
Prior predictive checking consists in simulating data from the priors:

- Visualise priors, especially after transformation.
- This shows the range of data compatible with the model before any data are seen.
- It helps understand the adequacy of the chosen priors, as it is often easier to elicit expert knowledge on measurable quantities of interest rather than abstract parameter values.

Data

Assume that the true data comes from the model

$$y_i = \alpha + \beta x_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2).$$



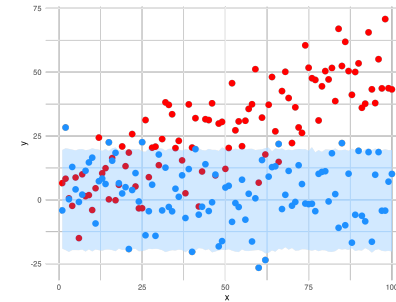
Model

We implemented the model in (one of) our favourite PPLs:

```
1 code <- nimbleCode({
2   alpha ~ dnorm(0, sd = 100)
3   beta ~ dnorm(0, sd = 100)
4   sigma ~ dunif(0, 10)
5
6   for (i in 1:n) {
7     y[i] ~ dnorm(alpha, sd = sigma)
8   }
9 })
```

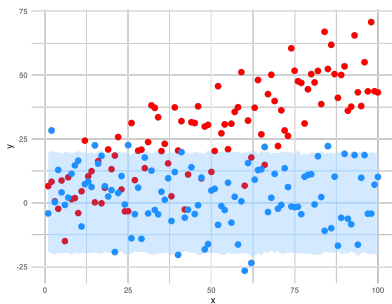
Prior predictive check

Let us draw samples from the priors, i.e. we are **not using any data at this stage yet**, only trying to see what kind of data (y) this model is able to generate.



Prior predictive check

Something doesn't look right...

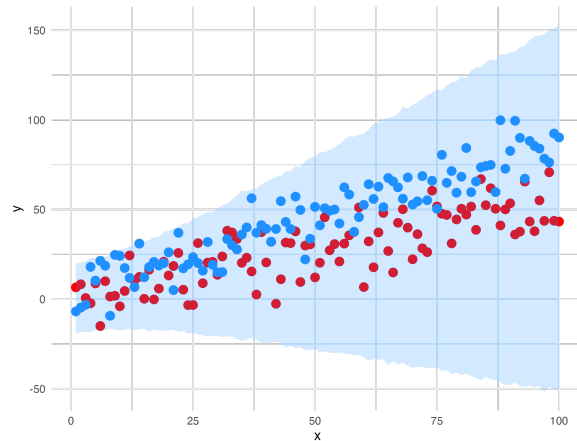


Debug

There was a bug in the model: the linear predictor left out the exposure entirely, so the model could only ever produce a flat line. Let's correct it:

```
1 code <- nimbleCode({
2   alpha ~ dnorm(0, sd = 100)
3   beta ~ dnorm(0, sd = 100)
4   sigma ~ T(dnorm(0, sd = 10), 0, Inf) # half-normal: truncated below at zero
5
6   for (i in 1:n) {
7     y[i] ~ dnorm(alpha + beta * x[i], sd = sigma) # the exposure term was missing
8   }
9 })
```

Prior predictive check again



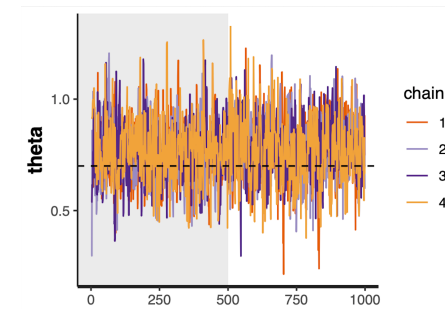
Better: now the range of prior predictive draws is covering the data.

Diagnosing MCMC outputs

- Convergence diagnostics
 - $\hat{R}$ statistic,
 - traceplots.
- Effective sample size (ESS):
 - Samples will be typically autocorrelated within a chain, which increases the uncertainty of the estimation of posterior quantities.
 - ESS – number of *independent* samples required to obtain the same level of uncertainty as from the available dependent samples.

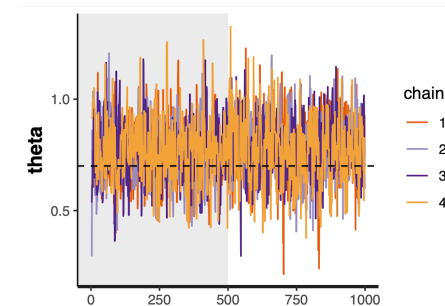
Posterior checks (MCMC output)

We use multiple chains of MCMC to estimate the posterior. In the lab we run four chains, each started from different initial values:



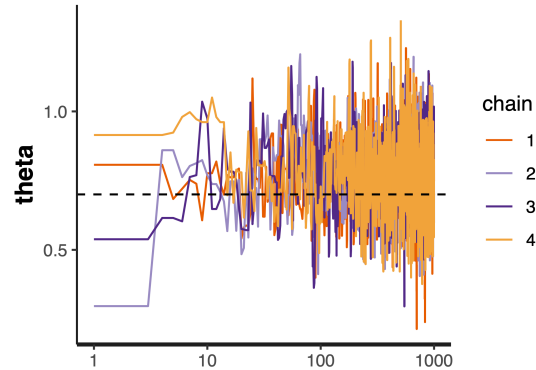
Diagnosing MCMC outputs

We use multiple chains of MCMC to inspect convergence after warm-up:



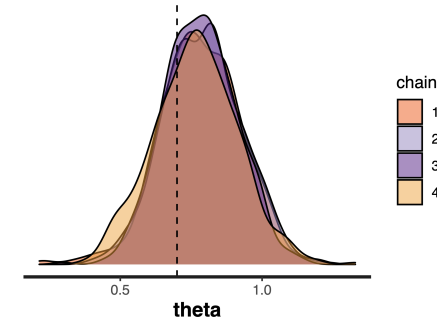
Diagnosing MCMC outputs

These chains have not converged: they are exploring different parts of the parameter space, which is exactly what a large $\hat{r}$ detects.



Diagnosing MCMC outputs

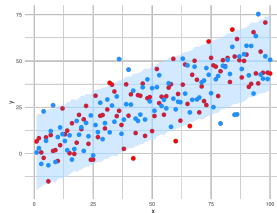
The post-warm-up samples of θ approximate its **posterior** distribution:



Posterior predictive checks

This examines how well your model fits the data.

- The idea is to compare the posterior of the fitted values with the observed values of your data.
- You can try different things: correlation, mean squared error, bias, coverage probability.



Sensitivity analysis

Having chosen priors and fitted a model, the last question of the workflow is whether the answer depends on the choices we made.

- Refit the model with reasonable alternative priors.
- Compare the posteriors for the parameters you actually care about.
- If they barely move, the data are informative and the priors are doing little work.
- If they move a lot, your conclusions are being driven by the prior, and that needs justifying rather than hiding.

Other

- Cross validation – The idea is to act like some of your data is missing, use your model to predict it, and then compare what the model predicts *well* with the truth.
 - How do you select the *missing* data and what does *well* mean?
- Information criteria – Penalise complexity over the fit. The smaller the better.
 - Examples: DIC, WAIC, AIC, BIC, etc.
- Deviance + effective number of parameters – the quantity most information criteria are built from.

The lab for this session

- This lab will take the workflow above and walk through it end to end in [NIMBLE](#), using simulated data so that we always know the right answer.
- During this lab session, we will:
 1. Simulate data from a known data-generating process;
 2. Perform prior predictive checks, including diagnosing priors that are too wide and too narrow;
 3. Fit a first model with MCMC and assess convergence using trace plots, rhat, and effective sample size;
 4. Perform posterior predictive checks and identify where the model is misspecified;
 5. Improve the model and compare results across specifications; and
 6. Run a sensitivity analysis to see whether our conclusions depend on the priors.

Getting ready for the lab

The lab for this session

- The [NIMBLE](#) mechanics are the same as this morning: priors first, then the formula, four chains from different starting values, then rhat.
- What is new is the loop around the fit.
- The first model we fit will be *deliberately wrong*, and the point of the lab is to find that out from the checks rather than from being told.

Questions?